TASK 10: Compare Expectations with Execution

Name: Nagineni Sunil Kumar

Register Number: 192472330

1. Aim

To compare floating-point and integer-paise fare totals for different journey volumes and analyse rounding errors, integer overflow, and compiler warnings.

2. C++ Program

```
#include <iostream>

#include <iomanip>

#include <climits>

using namespace std;

int main() {

   long long counts[] = {20000, 500000, 3000000};

   const long long farePaise = 1010;

   const double fareRupees = 10.10;

   cout << fixed << setprecision(2);

   for (long long n : counts) {

      double totalDouble = 0.0;

      long long totalPaise = 0;

      for (long long i = 0; i < n; i++) {

         totalDouble += fareRupees;

         totalPaise += farePaise; }
```

```
    double integerTotalRupees = totalPaise / 100.0;
    cout << "Journeys: " << n << '\n';
    cout << "Double total: Rs " << totalDouble << '\n';
    cout << "Integer-paise total: Rs "
        << integerTotalRupees << '\n';
    cout << "Difference: Rs "
        << totalDouble - integerTotalRupees << "\n\n"; }
  long long networkTotalPaise = 3000000LL * farePaise;
  cout << "INT_MAX: " << INT_MAX << '\n';
  cout << "Network total in paise: "
     << networkTotalPaise << '\n';
  if (networkTotalPaise > INT_MAX)
    cout << "The network total exceeds the typical int range.\n";
  return 0;}
```

3. Expected Mathematical Totals

Journeys	**Total at ₹10.10 per journey**
20,000	₹202,000.00
500,000	₹5,050,000.00
3,000,000	₹30,300,000.00

The integer-paise totals should match these exact amounts. The displayed double total and difference depend on the compiler and floating-point calculations; record the actual values produced by your own execution.

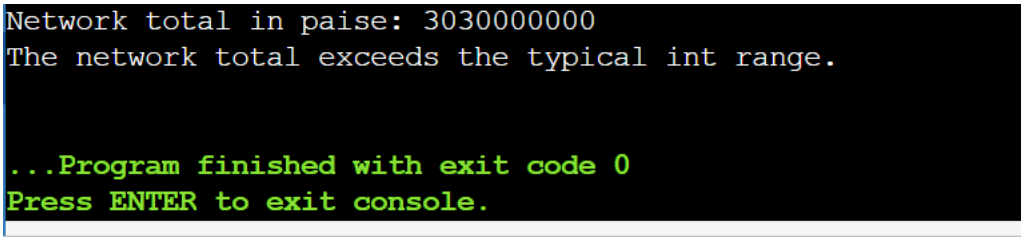
4. Integer Overflow Analysis

For 3,000,000 journeys:

[
3{,}000{,}000 \times 1010
= 3{,}030{,}000{,}000 \text{ paise}
]

A typical 32-bit signed int has a maximum value of 2,147,483,647. Therefore, the total in paise exceeds that range.

Signed integer overflow in C++ is undefined behaviour. Do not deliberately execute an overflowing signed int calculation. Use long long and check the range before performing potentially overflowing operations.



5. Compiler Warnings

Compile with:

g++ -std=c++17 -Wall -Wextra Task10.cpp -o Task10

These options enable useful warnings, but they do not guarantee that every precision problem or overflow risk will be reported. Record the warnings actually shown by your compiler. If there are no warnings, state that no warnings were displayed.

6. Alternative Concept and Complexity

- int: Insufficient for the example network total in paise.
- long long: Suitable for this total, provided operations remain within its range.
- double: Supports fractional calculations but can represent some decimal values only approximately.

Each addition takes O(1) time. Processing (n) journeys takes O(n) time. Each accumulator uses O(1) space.

7. Conclusion

Integer paise with long long provides exact monetary totals for this example. The comparison must be based on actual execution results, and the network total must not be accumulated in an insufficient integer type.